Genetic Algorithm For Special Shortest Path Routing

By: Tafadar Soujad

Abstract:

Dijkstra's algorithm has been proven to be the fastest known single-source shortest-path algorithm for arbitrary directed graphs with unbounded non-negative weights. The genetic algorithm schema is a method that evolves a group of stochastically produced solution candidates to try to obtain the optimal solution to a problem within a given range of convergence. In this paper, we will show how the limitations of Dijkstra's Algorithms can be expressed in a real world network routing problem. We proceed to show how an implementation of this situation can be represented by a graph, and how Dijkstra's Algorithm fails to converge at all when used on this graph, and how a Genetic Algorithm can succeed where Dijkstra's fails.

Introduction:

In many network routing problems dealing with the flow of information, the concept of a path having a negative weight to anybody familiar with computer networking can initially come off as nonsensical. However, with some creative intuition, we see that the inclusion of negative weights on a graph can be derived from a powerful basis that focuses on how to optimize the flow of information in a network. This takes the form of a Quality of Service routing structure which takes into account various factors on top of the traffic flow when deciding how an edge should be weighted, such as network latency and throughput[1], which is especially important in a modern routing scenario for quality dependent applications including real time multimedia streaming and and network gaming. The addition of these factors into the generation of a set of edge weights in a network graph can result in a negative edge weight, for example when a certain edge has a high hop count but results in a much lower latency or improved throughput leading to a negative weight given how the architects of the routing schema may value these other factors.

Dijkstra's algorithm is a method for finding the shortest paths in a graph with non-negative edge weights. It does so by iteratively exploring nodes from a chosen starting point, or source node. It goes on to evaluate the distances to all other nodes in the graph, and continuously updates them if it discovers a shorter path. The algorithm maintains a priority queue of nodes to explore further, and always selects the one with the smallest known distance. This continues until it reaches the desired destination or exhausts all reachable nodes when exploring the entire network like we show in this paper. Dijkstra's algorithm is efficient for finding shortest paths in graphs with relatively few edges and is widely used in various applications such as network routing protocols, navigation systems, and traffic planning.

A genetic algorithm is a heuristic optimization technique which simulates the biological processes of natural selection. It does this by stochastically generating and iteratively evolving a population of candidate solutions, called individuals or chromosomes. Each individual represents a potential solution and is encoded as a string of values, called genes. The algorithm evaluates the fitness of each individual based on how well it solves the problem according to some chosen metric by the architect of the algorithm implementation, and individuals with the highest fitness values are selected for reproduction. Through a process of selection, crossover, and mutation, new individuals are generated, combining traits from the fittest individuals in the population, less fit individuals are eliminated, and new individuals are stochastically generated to replace them. Over successive generations, the population evolves, and the solutions tend to improve. Genetic algorithms are widely used in optimization and search problems, offering a flexible and powerful approach to finding good, but not necessarily strictly optimal, solutions.

Here we provide an exploration on how Dijkstra's algorithm performs in such a schema, and, if this is insufficient, how a Genetic Algorithm can do better in this field.

Methodology:

Initially we implemented Dijkstra's algorithm in Python to provide a baseline for our experiment. This involved using a priority queue to iteratively explore the shortest path from the source node, which in our experiment we chose as node 'A', to each other node in the graph. This gave us a clear baseline of comparison for the network with only non-negative edge values as shown in Figure 1.

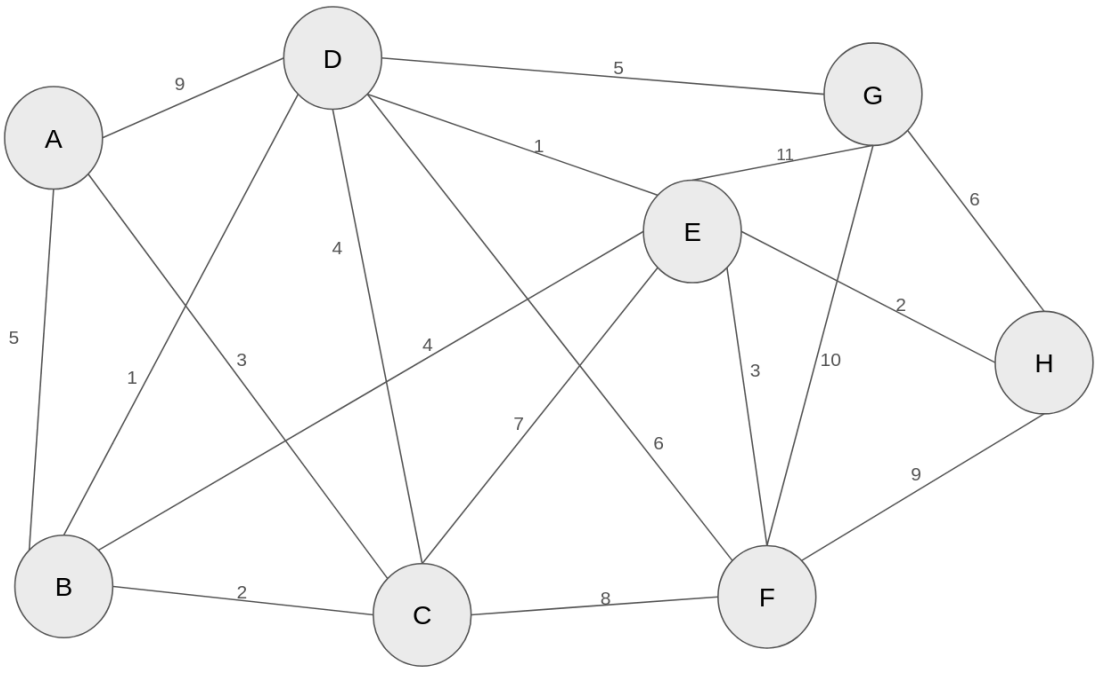

Figure 1: Implemented Network with Only Non-Negative Weighted Edges

From these results we are able to later compare our implementation of the genetic algorithm with the shortest paths found here. However, when we attempt to implement this same method with a network with edges that have negative values, we see that this algorithm fails to converge.

Further, we go on to implement a genetic algorithm to approach this shortest path problem. This involves creating a class to encapsulate the whole process which takes in an input of the source node, the destination node, the network represented as a graph in the form of a multi-layer dictionary, the number of generations to run the genetic algorithm for, and the initial population size of the individuals. We decided that this implementation did not require a mutation operator as it was witnessed that a mutation operation didn't necessarily increase the functionality of the algorithm.

Initially, the population is initialized to a specified population size by creating valid individuals, which means that there are only candidate solutions that are possible paths for the given network, as opposed to candidate solutions that contain sub-paths that don't exist(i.e. A -> E -> H). We then get the fitness for the candidate solutions by getting the total weight of the path represented by said candidate solution.

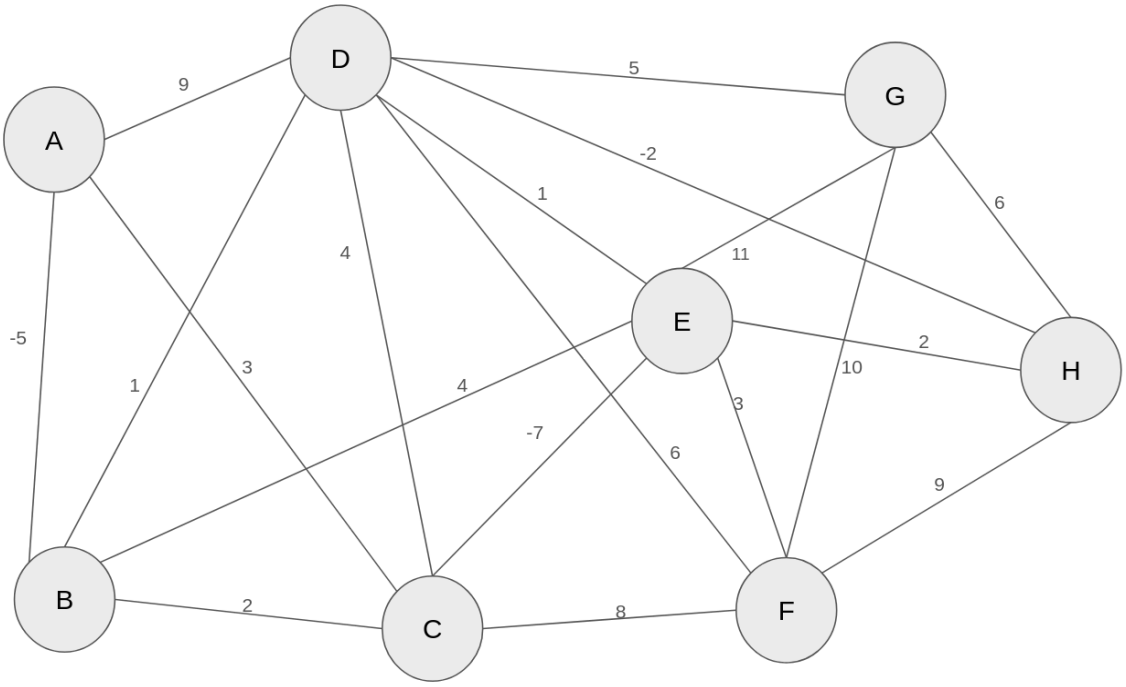

Figure 2: Implemented Network with Some Negative Weighted Edges

Afterwards the candidate solutions are ranked and the bottom half of the population is eliminated while the top half is kept for further generations. The parent candidate solutions, composed of every two candidate solutions of the remaining ranked population, are then recombined in such a way that the child solutions are kept to be valid individuals before adding them to the population now unranked. This begins the process of calculating fitness, ranking, eliminating, and recombining, which is iteratively done 10 times, said to be 10 generations. After this is done we extract the top ranked solution in the population as our overall solution to the problem

The code and direct results for this implementation of our methods are in a JupyterNotebook at: https://github.com/trs-code/genalgrouting

Results and Analysis:

We see in our implementation of Dijkstra's algorithm towards the network in Figure 1 that an optimal path can be found from source node 'A' to destination node 'H' with an optimal cost of 9, and this knowledge was achieved by Dijkstra's algorithm in an astounding recorded 0.03 milliseconds. However, when we try to implement this algorithm towards the network with some negative edges found in Figure 2, we see an immediate and start contrast in effectiveness, with an inability for the algorithm to cope and get stuck in a loop that must be manually exited by the experimenter with a keyboard interrupt command in the code.

Further we go on to test the genetic algorithm. We can usually get the optimal solution of (A -> B -> D -> E -> H) with a cost of 9 as seen ascertained in Dijkstra's solution. We also sometimes get the solution of (A -> C -> B -> D -> E -> H), also with a cost of 9, and less often the solution of (A -> C -> D -> E -> H) with a cost of 10. So we then implement the network with negative edge weights and try to derive the optimal solution for it since we cannot see the optimal solution from Dijkstra's algorithm. We then see that the optimal solution is likely (A -> B -> C -> D -> E -> H) with a cost of -11 as we continually get this top ranked solution when we run the genetic algorithm, however this may not be the optimal solution. We see that running this in both cases takes slightly over a millisecond, which is about 33x slower than the case of the graph with non-negative edge weights, and infinitely faster than in the case with some negative edge weights.

Thus we can see that even though this method is not as fast as Dijkstra's method, it can still hold up to take on various implementations that Dijkstra's method cannot measure up to.

Conclusion and Further Work:

We conclude from our exploration that while Genetic Algorithms are much slower than Dijkstra's algorithm, they are still able to work around certain limitations that are present in other common routing algorithms with a sufficient performance We see in [2] that a latency dependent application such as gaming can withstand a 40-60ms latency delay before being noticeable enough to hinder the user, which gives us much leeway regarding the ~1ms delay presented by the genetic algorithm.

The nice thing about genetic algorithms is that their time complexity does not scale up as the network becomes more populated, however the radius of convergence of the optimal solution does expand, so the complexity stays the same but the derived solution may be farther away than what we could ascertain with a thorough search such as in Dijkstra's. We can do further research to see how this would compare for a very large, sparse graph.

Furthermore, there is further research to see how this method compares to other algorithms, such as Bellman-Ford. The Bellman-Ford algorithm is typically known to be slower than Dijkstra's algorithm and can handle negative weighted edges, but it can be unsuccessful in cases where there is a cycle of nodes whose links all have negative weights, such as a source node connected to an intermediary node with a negative link weight, which is connected to the destination node through a link with a negative weight, that is also connected to the source node through a link with a negative weight. We can do further experimentation to see if a genetic algorithm can handle this scenario where a Bellman-Ford algorithm would likely face a fate similar to Dijkstra's algorithm taking on a network with links that have negative weights.

References

[1]Chen S., Narsdeht K., “An Overview of Quality of Service Routing for Next-Generation High-speed Networks: Problems and Solutions”, *IEEE Network*, November/December 1998

[2] M. Claypool and D. Finkel, "The effects of latency on player performance in cloud-based games," *2014 13th Annual Workshop on Network and Systems Support for Games*, Nagoya, Japan, 2014, pp. 1-6